

Python Notes

Memory, Data Structures & Logic Patterns

These notes cover how Python **actually** stores and manipulates data: the reference model behind variables, the complete behavior of list slicing, the three levels of copying, tuple unpacking, and the core string toolkit. Every trap here is something Python does **silently** — each example shows the exact output, so the goal is to **predict** behavior, not memorize it.

- | | |
|---|---------------------------------------|
| 1. Variables, References & Aliasing | 6. Tuples & Unpacking |
| 2. Mutability, Immutability & Hashability | 7. String Fundamentals & Memory Rules |
| 3. List Slicing — Assignment Mechanisms | 8. Text Cleaning & Formatting |
| 4. List Slicing — Traps & Edge Cases | 9. Splitting, Joining & Validation |
| 5. The Copy Concept: Alias / Shallow / Deep | |

1. Variables, References & Aliasing

1.1 Variables are references, not boxes

A Python variable is **not a box that stores a value** — it is a **name (label) that points to an object** living in memory. Assignment binds a name to an object; it never copies the object. The built-in `id()` function returns an object's unique identity (its memory address in CPython).

```
x = [1, 2, 3]
y = x                # y is bound to the SAME object, nothing is copied
print(id(x) == id(y)) # True -> one object, two names
```

1.2 Aliasing

When two names point to the same **mutable** object, they are **aliases**. A change made through one name is visible through the other — because there is only one object.

```
a = [10, 20, 30]
b = a                # aliasing: no new list is created
b.append(40)         # mutate through b ...
print(a)             # [10, 20, 30, 40] ... and a sees it too!
print(a is b)        # True

b = [99]             # REBINDING: b now points to a brand-new list
print(a)             # [10, 20, 30, 40] -> a is unaffected
```

KEY POINT: Mutation (`append`, `b[0] = ...`) travels through every alias. Rebinding (`b = new_object`) only moves the label — it disconnects the alias.

1.3 Integer interning (−5 to 256)

CPython pre-creates all integers from **−5 to 256** at startup and reuses those single objects everywhere. Numbers outside this range get a fresh object each time they are created separately.

```

a = 100
b = 100
print(a is b)      # True  -> both names reuse the one cached 100 object

x = 1000
y = 1000           # created separately (e.g. two lines in the REPL)
print(x is y)      # False -> two different objects with equal value
print(x == y)      # True

```

TRAP: The exact result of `is` on large numbers can vary (script vs REPL, constant caching). That unpredictability is the whole lesson: **never compare numbers with `is` — always use `==`**.

1.4 Identity vs equality: `is` vs `==`

`==` asks “do these objects hold **equal values**?” `is` asks “are these **the same object** at the same memory location?”

```

p = [1, 2]
q = [1, 2]
print(p == q)      # True  -> same values
print(p is q)      # False -> two separate list objects

r = p
print(p is r)      # True  -> same object (alias)

```

Strings behave like small integers: short, identifier-like string literals are **interned** (cached and reused), so `is` may return `True` for them — but this is an optimization, not a promise.

```

s1 = "hello"
s2 = "hello"
print(s1 is s2)    # True  -> interned (looks like a name/identifier)

s3 = "hello world!"
s4 = "hello world!"
print(s3 == s4)    # True
print(s3 is s4)    # False (typically) -> spaces/punctuation: not guaranteed

```

KEY POINT: Use `==` for value comparison. Reserve `is` for identity checks like `x is None`.

2. Mutability, Immutability & Hashability

2.1 Mutable vs immutable types

A **mutable** object can be changed in place after creation; an **immutable** object can never change — any “edit” must build a new object.

Category	Types	In-place change?
Mutable	list, dict, set, bytearray	Yes — the object itself is modified
Immutable	int, float, bool, str, tuple, frozenset, bytes	No — a new object must be created

```

lst = [1, 2, 3]
lst[0] = 99          # OK: lists are mutable
print(lst)           # [99, 2, 3]

t = (1, 2, 3)
t[0] = 99
# TypeError: 'tuple' object does not support item assignment

```

TRAP: A tuple is immutable, but it stores **references**. If a tuple contains a list, that inner list can still be mutated: `t = (1, [2, 3]); t[1].append(4)` gives `(1, [2, 3, 4])`. The tuple never changed — it still points to the same list; the **list** changed.

2.2 The hashability rule (dictionary keys)

A dictionary key must be **hashable**, which in practice means **immutable**: strings, numbers, and tuples (of immutables) work; lists do not.

```

d = {"name": "Asha", (1, 2): "point"}    # str key OK, tuple key OK

d[[1, 2]] = "list-key"
# TypeError: unhashable type: 'list'

d[(1, [2])] = "sneaky"                   # tuple containing a list?
# TypeError: unhashable type: 'list'    -> everything INSIDE must be immutable too

```

3. List Slicing — Assignment Mechanisms

Recap: `lst[start:stop:step]` reads from `start` up to **but not including** `stop`. Slices can also appear on the **left side** of `=` — that is slice **assignment**, and it modifies the list in place.

3.1 Simple slice assignment — flexible sizing

Without a step, the replacement does **not** need to match the slice length. The list simply shrinks or grows.

```

lst = [1, 2, 3, 4, 5]
lst[0:2] = [99]          # 2 items out, 1 item in -> list SHRINKS
print(lst)               # [99, 3, 4, 5]

lst = [1, 2, 3, 4, 5]
lst[0:2] = [7, 8, 9, 10]  # 2 items out, 4 items in -> list GROWS
print(lst)               # [7, 8, 9, 10, 3, 4, 5]

```

3.2 Extended slice assignment — strict sizing

The moment a **step** is involved (e.g. `[::2]`), Python demands an **exact size match** between the target positions and the values supplied.

```

lst = [1, 2, 3, 4, 5, 6]
lst[::2] = [10, 20, 30]  # targets indexes 0, 2, 4 -> exactly 3 values: OK
print(lst)               # [10, 2, 20, 4, 30, 6]

lst[::2] = [10, 20]
# ValueError: attempt to assign sequence of size 2 to extended slice of size 3

```

3.3 Empty slice insertion

If `start == stop`, the slice selects **nothing** — so nothing is removed and the new items are **inserted** at that position.

```
lst = [1, 2, 3, 4]
lst[2:2] = [99, 100]      # insert at index 2, remove nothing
print(lst)                # [1, 2, 99, 100, 3, 4]
```

3.4 Deletion through slicing — the full toolkit

Two routes: assign an **empty list** to the slice, or use the `del` statement. `del` also handles a single index and even extended slices. (Bare `del lst` deletes the **name** itself — using it afterwards raises `NameError`.)

```
lst = [1, 2, 3, 4, 5]
lst[1:3] = []             # route 1: assign empty list -> indexes 1, 2 removed
print(lst)                # [1, 4, 5]

lst = [1, 2, 3, 4, 5]
del lst[1:3]              # route 2: del with a slice -> same result
print(lst)                # [1, 4, 5]

lst = [1, 2, 3, 4, 5]
del lst[0]                # del with a single index
print(lst)                # [2, 3, 4, 5]

lst = [1, 2, 3, 4, 5]
del lst[::2]              # del with an EXTENDED slice: removes indexes 0, 2, 4
print(lst)                # [2, 4]
```

TRAP: With a step, the empty-list route breaks: `lst[::2] = []` raises `ValueError` (strict sizing — rule 3.2). But `del lst[::2]` works perfectly. For extended slices, **`del` is the only clean way to delete**.

3.5 Emptying a list — clearing vs rebinding

Three ways empty the **existing object in place**; a fourth look-alike only moves the label. With aliases around (Section 1.2), the difference is huge.

```
a = [1, 2, 3]
b = a                    # alias: one object, two names

a[:] = []               # in-place empty (same effect: del a[:] or a.clear())
print(b)                 # [] -> the alias sees it: the object is truly empty

a = [1, 2, 3]
b = a
a = []                  # REBINDING: 'a' now points to a brand-new empty list
print(a)                # []
print(b)                # [1, 2, 3] -> the old object is still alive through b!
```

KEY POINT: `lst[:] = []`, `del lst[:]` and `lst.clear()` empty the shared object — every alias sees it. `lst = []` only rebinds the name; other aliases keep the old data.

3.6 The iterable requirement

The right side of a slice assignment must be an **iterable**. A bare number fails — and a string, being iterable, spreads out **character by character**.

```
lst = [1, 2, 3]
lst[0:1] = 99
# TypeError: can only assign an iterable

lst[0:1] = [99]          # wrap it in a list -> OK
print(lst)              # [99, 2, 3]

lst = [1, 2, 3]
lst[0:1] = "hi"         # a string IS iterable...
print(lst)              # ['h', 'i', 2, 3] -> characters, one by one!
```

4. List Slicing — Traps & Edge Cases

4.1 Out-of-bounds forgiveness

Indexing outside the list raises an error, but **slicing** never does — Python silently clips the range to the list's actual end.

```
lst = [1, 2, 3]
print(lst[1:100])      # [2, 3] -> no error, clipped to the end
print(lst[50:100])     # [] -> still no error

lst[5:100] = [9]       # works in assignment too: lands at the end
print(lst)             # [1, 2, 3, 9]

print(lst[100])
# IndexError: list index out of range -> indexing stays strict!
```

4.2 The copy trap (chained assignment)

A slice on the **right side or in the middle** of an expression creates a **new temporary list**. Assigning into that temporary changes the copy — which is instantly thrown away.

```
lst = [1, 2, 3, 4]
lst[0:2][0] = 99       # lst[0:2] builds a TEMPORARY list [1, 2]
print(lst)             # [1, 2, 3, 4] -> original completely untouched!
```

4.3 The shallow copy illusion

`lst[:]` copies only the **outer** list. The inner (nested) lists are **not copied** — both outer lists keep pointing at the same inner objects.

```

a = [[1, 2], [3, 4]]
b = a[:]          # new outer list, SHARED inner lists

b[0][0] = 99      # reach inside through b ...
print(a)          # [[99, 2], [3, 4]] -> a is hit too!
print(a is b)     # False -> outer lists differ
print(a[0] is b[0]) # True -> inner lists are the same object

b.append([5, 6])  # outer-level change: only b grows
print(a)          # [[99, 2], [3, 4]]

```

4.4 The direction mismatch trap

Start and stop define a left-to-right window, but a negative step walks **right-to-left**. If the direction contradicts the window, Python returns an **empty list** — no error, no warning.

```

lst = [1, 2, 3, 4, 5]
print(lst[0:4:-1]) # [] -> start is LEFT of stop, but step goes LEFT: dead end
print(lst[4:0:-1]) # [5, 4, 3, 2] -> window and direction agree

```

4.5 The self-assignment rule

In `target = source`, the **source (right side) is fully evaluated first**. That is why a list can safely be built from itself.

```

lst = [1, 2, 3, 4]
lst[:] = lst[::-1] # right side becomes [4, 3, 2, 1] BEFORE assignment begins
print(lst)         # [4, 3, 2, 1] -> reversed in place, no corruption

```

4.6 The None magic trick

A blank position in a slice is internally `None`. Writing the Nones explicitly gives the **identical** result.

```

lst = [1, 2, 3, 4, 5]
print(lst[::-1])      # [5, 4, 3, 2, 1]
print(lst[None:None:-1]) # [5, 4, 3, 2, 1] -> exactly the same slice

```

5. The Copy Concept: Alias vs Shallow vs Deep

Trap 4.3 showed that `[ : ]` is only skin-deep. Here is the complete picture — three levels of “copying”, from none to full.

```

import copy
original = [[1, 2], [3, 4]]

alias    = original          # level 0: just another name
shallow  = original[:]       # level 1: also original.copy() / copy.copy()
deep     = copy.deepcopy(original) # level 2: copies every level

original[0][0] = 99
print(alias)    # [[99, 2], [3, 4]] -> sees every change
print(shallow)  # [[99, 2], [3, 4]] -> inner lists shared, so hit as well
print(deep)     # [[1, 2], [3, 4]]  -> fully independent

```

Technique	New outer list?	New inner objects?
<code>b = a</code> (alias)	No	No
<code>a[:], a.copy(), copy.copy(a)</code> (shallow)	Yes	No — shared
<code>copy.deepcopy(a)</code> (deep)	Yes	Yes — all levels

KEY POINT: Flat list? A shallow copy is enough. Nested list? Only `copy.deepcopy()` gives true independence.

6. Tuples & Unpacking

6.1 The single-element tuple trap

Parentheses alone do **not** make a tuple — the **comma** does.

```
t = (5)
print(type(t))    # <class 'int'>    -> just a number in brackets!
t = (5,)
print(type(t))    # <class 'tuple'> -> the comma makes the tuple
```

6.2 Basic unpacking & the Pythonic swap

Unpacking spreads the items of an iterable across names in one step. The count on both sides must match exactly. Because the **right side is evaluated first** (rule 4.5 again!), swapping needs no temp variable.

```
point = (3, 7)
x, y = point      # works with lists and strings too
print(x, y)       # 3 7

a, b = 1, 2
a, b = b, a        # right side becomes the tuple (2, 1) first
print(a, b)       # 2 1

a, b = (1, 2, 3)
# ValueError: too many values to unpack (expected 2)
```

6.3 Star unpacking

One name may carry a ***** — it absorbs “whatever is left” and is **always a list**, even if it catches nothing.

```
first, *middle, last = [1, 2, 3, 4, 5]
print(first)        # 1
print(middle)       # [2, 3, 4]    -> always a list
print(last)         # 5

a, *rest = [1]
print(a, rest)      # 1 []         -> the star can absorb zero items
```

7. String Fundamentals & Memory Rules

7.1 The immutability trap

A string can never be edited in place — not even one character. Every “change” must build a **new string**.

```
word = "hello"
word[0] = "M"
# TypeError: 'str' object does not support item assignment

word = "M" + word[1:]    # build a NEW string instead
print(word)              # Mello
```

7.2 Escape characters & raw strings

An escape like `\n` or `\t` is **one single character**, which confuses lengths and silently corrupts Windows-style paths. A **raw string** `r"..."` switches escapes off.

```
s = "a\nb"
print(len(s))    # 3  -> \n counts as ONE character
print(s)         # a
                 # b

path = "C:\notes\table.txt"  # \n -> newline, \t -> tab ... silently broken!
path = r"C:\notes\table.txt" # raw string: backslashes stay literal
print(path)                # C:\notes\table.txt
```

7.3 The multiplication trap

Multiplying a string by **zero or a negative number** is not an error — it quietly produces an **empty string**.

```
print("ab" * 3)    # ababab
print("ab" * 0)    # ''  -> empty string, NOT an error
print("ab" * -5)   # ''  -> negative behaves like zero
```

7.4 Method chaining

String methods return **new strings**, so they can be chained. Execution runs **left to right**: each method works on the result of the previous one.

```
text = "  HELLO World  "
result = text.strip().lower().replace("world", "python")
print(result)    # hello python
```

8. Text Cleaning & Formatting

8.1 The cleaners: `strip` / `lstrip` / `rstrip`

By default these remove **whitespace** from both ends, the left end, or the right end.


```
s = "  data  "
print(repr(s.strip()))    # 'data'
print(repr(s.lstrip()))   # 'data  '
print(repr(s.rstrip()))   # '  data'
```

TRAP: `strip("abc")` does **not** remove the exact word "abc". The argument is a **set of characters** — any combination of them is peeled off both ends until a different character appears.

```
s = "abcHELLOcba"
print(s.strip("abc"))      # HELLO    -> a, b, c stripped in ANY order

print("www.example.com".strip("wcom."))
# example    -> 'w', '.', 'c', 'o', 'm' peeled from both ends
```

8.2 The replacer trap

Because strings are immutable, `.replace()` **returns a new string** and leaves the original untouched. Forgetting to re-assign the result is one of the most common beginner bugs.

```
msg = "I like Java"
msg.replace("Java", "Python")    # result created ... and thrown away
print(msg)                      # I like Java    -> unchanged!

msg = msg.replace("Java", "Python") # assign it back to keep it
print(msg)                      # I like Python
```

8.3 Case normalizers

Four related methods, two of which are easy to mix up: `.title()` capitalizes **every word**; `.capitalize()` capitalizes **only the first word** (and lowercases the rest).

```
s = "python is fun"
print(s.upper())    # PYTHON IS FUN
print(s.lower())    # python is fun
print(s.title())    # Python Is Fun    -> every word
print(s.capitalize()) # Python is fun    -> only the first word
```

KEY POINT: `.title()` restarts capitalization after **any** non-letter: `"don't".title()` gives `"Don'T"`.

9. Splitting, Joining & Validation

9.1 The breaker: smart split vs strict split

`split()` with **no argument** is smart: it treats any run of whitespace as one separator and ignores the edges. `split(" ")` is strict: **every single space** is a cut point, so doubled spaces produce empty strings.

```
s = "a  b  c"
print(s.split())    # ['a', 'b', 'c']    -> smart
print(s.split(" ")) # ['a', '', 'b', '', 'c'] -> strict
```

9.2 The builder: join

`separator.join(iterable)` glues pieces into one string. Every piece must **already be a string** — numbers in the list raise a `TypeError`.

```
words = ["Python", "is", "fun"]
print(" ".join(words))          # Python is fun
print("-".join("abc"))          # a-b-c    -> any iterable of strings works

nums = [1, 2, 3]
print(",".join(nums))           # TypeError: sequence item 0: expected str instance, int found
# TypeError: sequence item 0: expected str instance, int found

print(",".join(str(n) for n in nums))  # 1,2,3    -> convert first
```

9.3 The validation trap: isalpha / isdigit / isalnum

These check **every character** — a single space, dot, or punctuation mark anywhere makes the whole answer `False`. On an empty string they are all `False` too.

```
print("hello".isalpha())        # True
print("hello world".isalpha())  # False  -> the space fails it
print("123".isdigit())          # True
print("12.5".isdigit())         # False  -> the dot is not a digit
print("abc123".isalnum())       # True
print("abc 123".isalnum())      # False  -> space again
```